

NERV

CODEFEST AD ASTRA 2026

Informe Técnico — Etapa 1

Equipo: NERV - Universidad Nacional de Colombia sede Manizales

Integrantes

Sergio Alejandro González Osorio
Andrew Esteban Henao Becerra
Mateo Alexander Cardenas Cuesta
Felipe Camacho García

1. Introducción

El presente documento describe las decisiones de diseño tomadas por el equipo NERV para la Etapa 1 del reto CODEFEST AD ASTRA 2026: la construcción de una base de conocimiento vectorial sobre un corpus documental correspondiente a tres fenómenos temáticos — (1) inteligencia artificial e innovación en entornos militares, (2) seguridad espacial y órbita baja terrestre, y (3) dinámicas territoriales en América Latina — distribuidos en formatos PDF, HTML, CSV, XLSX, JSON, imágenes y PBF.

El corpus procesado en la línea limpia actual alcanzó 335,371 chunks provenientes de 1,760 documentos distintos. El sistema resultante fue evaluado sobre las 50 consultas oficiales de la Etapa 1, cubriendo los tres fenómenos en español, inglés y portugués.

2. Preprocesamiento y extracción

El módulo de ingesta produce un contrato de salida único (`documentos.jsonl`) consumido por la etapa de chunking, con los campos `doc_id`, `fuentes`, `formato`, `fenomeno`, `idioma` y `texto`. El "fenomeno" se infiere automáticamente del corpus crudo, y `doc_id` se calcula mediante SHA-256 sobre la ruta relativa normalizada y el contenido binario del archivo, garantizando identidad documental estable y reproducible.

Extracción por formato: los PDF se procesan primero con extracción nativa de texto (PyMuPDF); OCR con Tesseract se activa únicamente cuando una página carece de texto extraíble, procesando a 220 DPI en español, inglés, portugués y chino simplificado. Se implementó una corrección crítica para PDFs mixtos: si alguna página requiere OCR, el documento completo se difiere a una segunda pasada para preservar el orden original. Además, si el OCR de una página retorna vacío, el documento se conserva siempre que existan otras páginas con texto útil, previniendo la pérdida de documentos válidos. Los archivos `.pbf` se interpretan como *Mapbox Vector Tiles*, extrayendo únicamente propiedades semánticas con deduplicación por huella de capa e identificador o, en su ausencia, por propiedades normalizadas.

Control de calidad: el sistema detecta contenido sin valor, como marcadores de página residuales, luego los elimina para evitar que afecte a los embeddings.

3. Estrategia de chunking y su justificación

La fragmentación combina detección automática de idioma (español, inglés, portugués) con segmentación de oraciones mediante PySBD, respetando en todo caso el requisito de completitud lingüística: ningún fragmento contiene oraciones cortadas. Para contenido tabular (CSV, TSV, XLS, XLSX) se preservan los límites estructurales de fila en lugar de aplicar segmentación de oraciones.

Dimensionamiento: los candidatos se miden con el tokenizador real del encoder (`intfloat/multilingual-e5-small`), con un objetivo blando de 256 IDs y superposición objetivo de 32 IDs entre unidades consecutivas. El prefijo "passage: " se incluye en el conteo pero no se persiste en el campo `texto` almacenado.

3.1 Corrección de límite duro: caso documentado (ADR)

Una línea base sobre el corpus real y auditorías posteriores identificaron fragmentos de contenido estructural (como CSV y JSON) que excedían el límite seguro de entrada del encoder (510 IDs totales). Este hallazgo — documentado formalmente en `docs/decisions/ADR-encoder-hard-limit-rev-a.md` — motivó una corrección aceptada del pipeline: se preservan como unidades completas los fragmentos hasta 510 IDs, y únicamente se subdividen aquellos que superan 510, aplicando primero separadores tabulares o de JSON, luego límites textuales (salto de línea, puntuación), y como último recurso, offsets exactos del tokenizador recontados. La truncación silenciosa quedó explícitamente prohibida: todo

registro final se recuenta y el pipeline rechaza publicar cualquier fragmento que se exceda demasiado.

Este hallazgo es relevante más allá de la correctitud de los datos: los fragmentos anómalos inflaban el costo de generación de embeddings por lote, al forzar el *padding* de todo el lote a la longitud del elemento más largo (véase Sección 7).

3.2 Auditoría de Completitud Lingüística

Una auditoría global reciente sobre los 335,371 chunks detectó 47,142 fragmentos de prosa incompleta. Se diagnosticó que la causa dominante no es el segmentador (PySBD) ni los cortes por límite duro, sino artefactos de extracción upstream originados en los PDFs (saltos de página y línea). Aunque se aplicó un parche de presentación visual en los resultados finales, el sistema requerirá una revisión de fronteras upstream y una regeneración futura del corpus para lograr el cumplimiento estricto de este requisito.

4. Codificación semántica: selección del encoder

Se seleccionó [intfloat/multilingual-e5-small](#) como encoder único para todo el corpus, evaluado contra los criterios de selección especificados en el reto:

- **Soporte multilingüe:** el corpus combina fuentes en español, inglés y portugués; E5 representa los tres idiomas en un espacio semántico compartido. En pruebas de recuperación, consultas en español recuperaron consistentemente fragmentos relevantes en portugués sin traducción explícita, confirmando esta propiedad empíricamente.
- **Dimensionalidad:** 384, balance entre expresividad semántica y costo de almacenamiento/búsqueda para un índice de más de 335,000 vectores.
- **Límite de entrada:** se fijó el límite efectivo de codificación en 256 tokens, alineado al objetivo blando de chunking, con una capacidad segura del encoder de 510 IDs según el ADR de la Sección 3.1.
- **Eficiencia computacional:** modelo compacto (*small*), permitiendo generar los embeddings de todo el corpus en aproximadamente 30 minutos usando una GPU T4 (véase Sección 7).
- **Licencia:** MIT, disponible públicamente en HuggingFace.

Siguiendo el contrato estándar de E5, se antepone "**passage:** " a cada fragmento indexado y "**query:** " a cada consulta antes de codificar; este prefijo afecta únicamente la entrada del encoder y no se persiste en el texto almacenado. Todos los vectores se normalizan a norma unitaria (L2).

5. Gestión del índice vectorial con FAISS

Se utilizó `IndexFlatIP` (búsqueda exacta por producto interno). Con vectores normalizados en el espacio L2, el producto interno es matemáticamente equivalente a similitud coseno, y para el volumen del corpus (~335,371 vectores de dimensión 384) la búsqueda exacta resultó computacionalmente viable, evitando renunciar a exactitud a favor de un índice aproximado (IVF/HNSW). El índice se persiste con `faiss.write_index()` y se recupera con `faiss.read_index()`, sin dependencias adicionales. La alineación entre el índice y el almacén de metadata (posición interna $i \leftrightarrow$ línea i de `metadata.jsonl`) se verifica programáticamente antes de construir el índice.

6. Módulo de recuperación

El flujo de recuperación obtiene hasta 50 candidatos por consulta desde FAISS, los deduplica por `chunk_id`, y construye dos vistas: el top-10 de fragmentos (ordenados directamente por puntuación) y el top-3 de documentos (mediante agregación). Ningún paso del pipeline utiliza modelos generativos, cumpliendo la restricción de la especificación.

6.1 Decisión de agregación documental: experimento CombSUM vs. max pooling

Se evaluaron dos estrategias de agregación a nivel documento. Con *CombSUM* (suma de puntuaciones de todos los fragmentos recuperados de un mismo documento), el documento dueño del fragmento individualmente más relevante coincidía con el top-3 de documentos en solo 25 de 50 consultas (50 %): documentos con muchos fragmentos (p. ej. conjuntos tabulares convertidos fila por fila) ganaban por volumen acumulado, no por relevancia puntual. Con *max pooling* (puntuación del documento = puntuación de su mejor fragmento), esa coincidencia se garantiza matemáticamente en el 100 % de los casos, ya que ningún documento puede superar en agregado al que contiene el fragmento de mayor puntuación global. Se adoptó max pooling como estrategia final.

6.2 Formato de salida

Cada fragmento del top-10 se trunca, cuando excede 250 palabras, respetando el último cierre de oración completo dentro del límite, conforme al requisito de la especificación. Un validador automático verifica, antes de escribir `resultados.jsonl`, que las 50 líneas existan en orden q001–q050, con exactamente 3 documentos y 10 fragmentos por consulta y ningún fragmento por encima del límite de palabras.

6.3 Control de calidad de datos: detección de contaminación

Durante la validación de resultados se detectó que un archivo interno de seguimiento del equipo (FASE ORDENADA CODEFEST.xlsx, ubicado dentro de la carpeta del fenómeno 3) se había incorporado accidentalmente al corpus de ingesta. Ocho de sus fragmentos contenían texto que citaba literalmente el enunciado de algunas de las 50 consultas oficiales junto a un fragmento de referencia ya elaborado, generando una similitud artificialmente alta entre esos chunks y sus consultas correspondientes — una fuga de datos de evaluación hacia el corpus de conocimiento, no una coincidencia semántica genuina.

Se implementó un filtro automático de contaminación con dos capas de defensa, ejecutado como etapa obligatoria del pipeline antes de la generación de embeddings: (1) exclusión por coincidencia exacta de la fuente del documento contra una lista de fuentes ya identificadas como contaminadas, y (2) detección de patrones estructurales del archivo (marcadores como PREGUNTA:, FRAGMENTO:, DOCUMENTO:) junto con coincidencia textual literal entre el contenido de un fragmento y el texto completo de alguna consulta oficial. Esta doble capa permite que el problema no dependa de detección manual si el mismo archivo, u otro con estructura similar, reaparece en una versión futura del corpus.

6.4 Capa Multiagente (Stage 2)

Posteriormente a la Etapa 1, se integró una capa multiagente utilizando NERV como backend de recuperación. Esta arquitectura incluye un `RetrieverAgent`, un `AnalystAgent` y un `VerifierAgent` para orquestrar respuestas razonadas basándose estrictamente en la evidencia recuperada. Esta fase opera de manera independiente y no afecta al flujo determinista y oficial del reto principal.

7. Resultados y desempeño

La diferencia entre generación de embeddings en CPU (estimación inicial de ~35 horas, extrapolada mediante calibración sobre una muestra) y en GPU (~33 minutos reales) motivó migrar esa etapa a un entorno con aceleración por hardware (Google Colab, GPU T4), manteniendo la construcción del índice y la recuperación en CPU local, dado su costo marginal. Al final, gracias a la disponibilidad de un computador con su propia GPU, fue posible ejecutar todo el programa de principio a fin, desde la ingesta hasta la generación de los resultados en aproximadamente 2 horas y 10 minutos.

Observaciones cualitativas: las consultas de prueba confirmaron recuperación cruzada de idioma consistente (p. ej. una consulta en inglés sobre clasificación de señales mediante IA recuperó, en segundo lugar, un fragmento en portugués semánticamente equivalente). Tras adoptar max pooling, persiste una

concentración moderada de un número reducido de documentos de gran tamaño en el top-3 de múltiples consultas distintas — mitigada pero no eliminada por completo, dado que un documento con muchos fragmentos conserva mayor probabilidad estadística de que al menos uno puntúe alto.

8. Conclusiones

El sistema construido establece una línea base trazable y reproducible (1,760 documentos y 335,371 chunks) que cumple con los lineamientos de la arquitectura de la Etapa 1. Se corrigieron de manera sistemática problemas de ingesta críticos en la extracción de PDFs y la contaminación accidental del corpus con material de evaluación. Sin embargo, se ha identificado mediante auditorías posteriores que el sistema actual aún no cumple de forma estricta con el requisito de completitud lingüística debido a fallas en la extracción original de PDFs upstream, definiendo así el trabajo futuro necesario para alcanzar la conformidad total antes de consolidar la entrega definitiva.

Anexo de actualizacion de la linea Stage 1

Estado tecnico consolidado al cierre de la publicacion de codigo fuente

Alcance de las cifras del informe base

Las paginas anteriores documentan la linea limpia de referencia de 1.760 documentos y 335.371 chunks, junto con la auditoria que encontro 47.142 chunks de prosa con continuidad confirmada. Esas cifras se conservan como evidencia historica y no son los conteos finales de la entrega corregida. El indice, metadata y resultados de esa linea no deben promoverse como artefactos corregidos.

Correccion determinista implementada

La segmentacion conserva la procedencia de saltos de linea y marcadores de pagina de PDF. Una reconciliacion en orden de fuente une solamente fronteras con evidencia determinista de continuacion: espacio continuo con inicio en minuscula o conector, envoltura de una linea entre prosa, marcador de pagina, rotulo corto seguido de prosa, fuente, enumeracion o pie. La puntuacion terminal permanece autoritativa y los campos JSON, filas CSV, tablas y rotulos independientes no se fusionan. Solo cambia el espaciado; no se inventan palabras ni puntuacion.

Completitud y seguridad del encoder

El empaquetado mantiene objetivo blando de 256 tokens, solapamiento de 32 tokens en unidades completas y maximo almacenado de 510 tokens. Registros estructurales CSV/JSON y listas o tablas reconocidas pueden subdividirse con trazas de unidad padre. Una unidad indivisible de prosa por encima de 510 falla cerrada con GENUINE_OVERSIZED_SENTENCE_BLOCKER; no existe truncacion silenciosa.

Evidencia previa a la ejecucion completa

Control	Resultado
Documentos examinados	1.760
Unidades reconstruidas	1.709.477
Candidatas con conteo exacto	42.661
Unidades sobre 510	497; todas estructurales
Prosa sobre 510 / casos ambiguos	0 / 0
Limites PDF conocidos	184 de 184
Continuaciones bajo espacio continuo	81 de 81

El preflight no genero chunks de produccion, embeddings, FAISS ni resultados. La ejecucion completa posterior fue controlada por el operador humano.

Criterios de aceptacion de la entrega

Los artefactos definitivos solo pueden copiarse a entrega/ desde una linea corregida cuyo manifiesto haya terminado y cuya auditoria sea CORRECTED_CHUNKS_PASS.

Gate de chunks

La auditoria exige 1.760 documentos de origen, orden estable, IDs unicos, recuento exacto, maximo 510, cero prosa incompleta, cero fronteras de continuacion, cero contaminacion y metadata valida para toda subdivision estructural. La ejecucion humana 20260816-233354-stage1-corrected-chunks termino con 341.690 chunks, maximo almacenado 510 y SHA-256

92BA1EACDB4FE66563625E37CD3D6271ADF413E5EA72364A358051B0DCBA70C0. Todavia no existe corrected_chunks_audit.json; estos son conteos de candidato completado, no una aceptacion final.

Gate downstream

Despues de aceptar chunks deben regenerarse embeddings desde el texto exacto corregido. Se validan filas iguales al numero de chunks, 384 dimensiones, valores finitos, norma L2, IndexFlatIP, ntotal y filas de metadata iguales al numero de chunks, e identidad ordenada entre vector, metadata y texto. La recuperacion oficial exige q001-q050, tres documentos y diez fragmentos por consulta, maximo 250 palabras, esquema y trazabilidad completos.

Aislamiento y reproducibilidad

Stage 1 usa intfloat/multilingual-e5-small, vectores de 384 dimensiones, prefijos exactos passage: y query:, normalizacion L2, FAISS IndexFlatIP y max pooling numerico. No usa agentes, OpenAI, NVIDIA ni modelos generativos. La capa multiagente descrita en la Seccion 6.4 pertenece exclusivamente a Stage 2 y no participa en el ejecutable oficial.

Estado del paquete

El codigo fuente, pruebas focalizadas, scripts con gates, generador determinista y fuente documental estan listos. index.faiss, metadata.jsonl y resultados.jsonl deben sustituirse por los artefactos corregidos cuando termine la validacion. No se declara NDCG@10 ni F1@3 oficial porque las etiquetas ocultas no estan disponibles.

Tiempos medidos

El chunking completo de la linea de referencia tomo 885,257 segundos. El candidato corregido completo chunking en 1.309,755 segundos; ese tiempo es evidencia de ejecucion, no de aceptacion ni de mejora de rendimiento. Los tiempos de aproximadamente 33 minutos para embeddings GPU y 2 horas 10 minutos para el flujo historico permanecen asociados a la linea anterior. Downstream corregido aun no tiene tiempo medido.